A headless workstation for local models and coding agents

Design note on `cdl-box`, for readers deciding whether to build it

Jeremy Manning

2 September 2026

The machine is already there, and it is mostly asleep

A laptop with a discrete GPU is a compromise in both directions. It is heavier and hotter than a laptop needs to be, and the GPU sits idle whenever the lid is shut, which is most of the time. A Lambda TensorBook with 64 GB of RAM and an RTX 3080 spends most of its life as an expensive way to read email.

We propose to stop treating it as a laptop: put it in the office, leave it on, reach it over SSH, and let it do three things continuously, namely serve local models to whatever device you are actually using, run coding agents in sessions that survive a dropped connection, and fine-tune models on a GPU that would otherwise be idle. In other words, the hardware is already bought (and mostly unused), so the change we are proposing is one of posture rather than of capability.

A second motivation may matter more than it sounds, which is that configuring a machine this carefully is something we would like to do once rather than repeatedly. Every step of the setup is therefore a line in a script rather than a memory of what worked, so that the machine can be rebuilt from that script after a failure, a wipe, or a decision to move to different hardware. For example, we declare the storage layout in an installer file that has been executed twice, rather than describing it in a runbook that someone follows by hand (which is the form that drifts without anyone noticing).

What it is, and what it is not

`cdl-box` is a minimal Ubuntu Server 26.04 install plus a provisioning script. It carries four agent CLIs (Claude Code, Codex, Gemini and OpenCode), Ollama and `llama.cpp` for serving models, a working CUDA and PyTorch stack for training, Tailscale and SSH for reach, and a shell environment configured to be pleasant rather than default.

It is deliberately **not** several things. It is not an agent orchestration system: agents run in `zellij` sessions and a human decides what to run. It is not a custom Linux distribution, because a re-runnable script buys the same reproducibility at a fraction of the cost. It is not multi-user, and its security model assumes one person at one machine. Finally, it is not a general-purpose desktop.

The decision that shaped everything else

The machine is headless. No compositor, no display manager, no session lock, no autologin, no graphical stack at all. The internal panel shows a console login and nothing more; SSH is the interface.

That single choice removed most of the project. An earlier design of ours carried a display session, and with it a screen-locking problem that had no clean answer (the kiosk compositor we were considering ships no session-lock protocol at all, and the upstream issue is still open), plus autologin, plus a recovery path for a crashed compositor. It also retired hibernation as a requirement, since an always-on server does not hibernate, and that in turn retired a conflict between Secure Boot and kernel lockdown (which had been blocking us for weeks).

Conceptually, what unlocked this was noticing that a machine nobody sits at does not need a screen, and that almost everything expensive in the original design existed to serve one.

How it works

Storage. We stripe the two 1 TB NVMe drives into one volume: `md0` RAID0, LUKS on top, then btrfs with subvolumes for the system, home, and model weights. `/boot` sits outside the encryption, because GRUB has to read a kernel before anything can be unlocked. Striping is a deliberate trade (we recommended against it, and were overruled for reasons we think are defensible), and the spec records both sides of it rather than only the conclusion.

Serving models. Ollama is the endpoint, on a fixed port bound to the Tailscale network, and it is what other devices point at. `llama.cpp`'s `llama-server` behind `llama-swap` is a second, separate endpoint on localhost, for the cases Ollama does not cover. There is no reverse proxy and no unified router, because inventing one is work with no payoff at this scale. A client picks a port and names a model; nothing guesses.

Running agents. Each agent CLI launches through a wrapper that builds its environment per process, and sessions run inside `zellij` so that they survive disconnection. We write a transcript to disk by default (rather than as an option), because losing the only record of what an agent did is the kind of loss that stays invisible until the moment it matters.

Reach. Tailscale for addressability, OpenSSH with keys only, and `mosh` for sessions over poor links. The model endpoint binds to the tailnet interface, never to every address.

A small web dashboard, served from the box over the tailnet, answers the questions a terminal is bad at: GPU temperature and VRAM, which model is resident, what sessions are running, disk, SMART status for both drives (which on a striped pair is the only early warning there will be), and when the last backup ran. We keep it read-only in version one, since every control is a way to break something from a phone.

Key decisions, and the evidence behind them

Several of these were settled by measurement rather than by argument, which we mention because it is not usually possible.

LM Studio is out. Its desktop headless mode, by its own documentation, *"works on Mac, Windows, and Linux machines with a graphical user interface"*, so it cannot run on this box. Its server-native daemon can, but installs by piping a URL into a shell with no stated licence, so it cannot be provisioned. Ollama and `llama.cpp` cover serving and are redistributable.

Provider credentials are built per process, never set globally. This follows from measured behaviour: Claude Code's phone-steering feature refuses to run when `ANTHROPIC_BASE_URL` points anywhere but the vendor endpoint. The obvious design (i.e. one local gateway, privacy variables on by default, set in `/etc/environment`) would silently remove a capability the user pays for.

Backups go to a Hugging Face bucket, through `rclone`. HF Storage Buckets are S3-compatible, and the Hub's documentation states the use case: *"Buckets are well-suited for maintaining rolling backups."* We tested it. `restic`'s own S3 backend **cannot** initialise a repository there, failing at `client.BucketExists: 400 Bad Request`, because it reads the account namespace as the bucket name. Routed through `rclone`, every step passes: initialise, back up, verify with `check --read-data`, restore with a byte-identical `diff`, and prune.

Thermal policy rests entirely on throttling, because this machine has no fan control at all, in firmware or in the OS. We measured that during a firmware walk (zero fan inputs, zero writable PWM channels), so the available levers are `intel_pstate`'s `no_turbo` and `max_perf_pct`, both of which we confirmed writable. We should say plainly that the thresholds we have chosen are guesses until they are measured under sustained load on the machine itself.

What to push on before we build this

Ordered by how much damage a wrong answer does. We would rather hear about these now.

1. Right now the machine has no redundancy and no protected backup, at the same time. RAID0 means either drive failing destroys the volume (both drives, not one). Separately, the HF bucket has no versioning and no lifecycle rules, so a write-capable token on the box can erase the backup history permanently and irrecoverably. Our answer is a second copy, pulled by a different machine for which the box holds no credential. We think that is a sound mitigation, and it is also (at present) the only thing standing between one bad command and total loss. What would settle it: build the second copy first, before anything else, and test recovering from it.

2. Every reboot needs someone at the machine. The encrypted root is unlocked by a passphrase typed at the console, and there is deliberately no remote unlock. In an office rather than at hand, that means a power cut on a Friday leaves an unreachable machine until some-

one drives in (and from outside, an unreachable machine looks identical to a dead one). A UPS would convert the commonest cause into nothing at all, and may be the cheapest available fix. The alternative, binding the key to the TPM, removes the trip but means that anyone who steals the whole machine and boots it gets the data.

3. The install is the least reproducible part of a design built around reproducibility. The provisioning script starts *after* the operating system is installed, so it reproduces everything except the storage layout, which is the part most likely to be got wrong. We have built a VM harness that performs the install unattended from a declarative file (twice, from clean) to close this. It runs on ARM while the target is x86-64, so the bootloader package differs, though the storage layer itself is architecture-independent and is the part we actually need to rehearse.

4. The GPU lock is cooperative, and cooperative locks are honoured only by the people who remember them. A wrapper takes a `flock` before serving or training, so a training run and a model load cannot both claim the card. Anything that runs a training script directly, without the wrapper, takes VRAM and the lock stops nothing. Making the wrapper the path of least resistance is the only enforcement available without containers, which are not worth their cost here.

5. "Budget" promises more than it delivers. The system decides whether an agent starts; it does not sit between an agent and its provider, so it cannot refuse an individual API call. A per-unit budget is a stop-loss rather than a cap, and overshoot is bounded by one request plus the detection interval rather than by zero.

6. Finally, the premise deserves a challenge. This machine is worth building if a GPU you can reach from anywhere, all the time, is genuinely more useful than a GPU in a laptop you open sometimes. If the answer turns out to be that the models you actually use are hosted ones (and that the local GPU is a hobby), then this may be a weekend of setup in service of a workflow that does not yet exist. What would settle it: build the first three milestones, use the machine over SSH for a week, and notice whether you keep going back to it.

Where this stands

The design is specified and none of it is built. Two of the riskiest questions have been answered by experiment rather than by reasoning: the backup path works, through a transport the design did not originally specify, and the storage layout is being rehearsed in a VM before any real disk is touched.

The next step is deliberately not more design. It is to back up the machine that exists today, prove the restore works, and only then repartition anything.

What we would like from a reader: a judgement on the premise, an opinion on whether striping two drives with no immutable backup is a trade you would make, and any failure mode in the boot or recovery path that we have not thought of.